SECURE E-VOTING SYSTEM

Introduction to Computer Security

Course	Introduction to Computer Security
Level	3rd Year Engineering — AI Specialization
Student 1	Brahmia Lokman Abdelmoname
Student 2	Meghlaoui Tarek
Student 3	Khennat Abdellah
Date	30/04/2026

Confidentiality • Integrity • Authenticity • Non-repudiation

1. Introduction

This project presents a secure electronic voting system based on a client-server architecture. The primary objective is to ensure that votes are transmitted securely, cannot be altered, and are only cast by authorized users.

The system addresses a fundamental challenge in cybersecurity: how to verify voter identity while maintaining vote confidentiality. By combining RSA encryption, digital signatures, and SHA-256 hashing, the system provides a robust and tamper-proof voting mechanism.

2. System Description and Architecture

2.1 General Overview

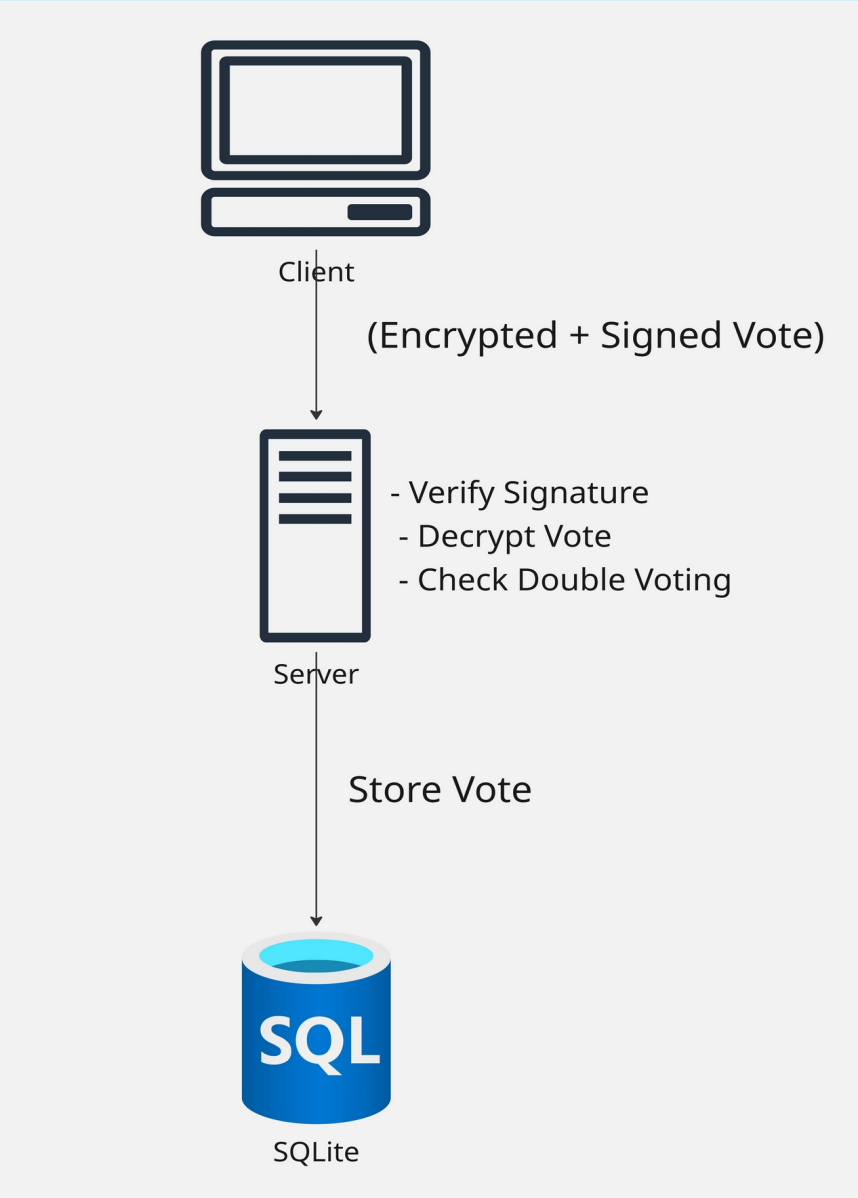
The system is divided into two main components:

- Client (GUI): Used by voters to submit their vote through an intuitive graphical interface.
- Server: Responsible for verifying signatures, decrypting votes, and counting results.

2.2 Voting Flow

1. The client requests the server's public key.
1. The voter selects a candidate.
1. The vote is encrypted using RSA.
- The encrypted vote is signed using the voter's private key.
- The client sends the signed, encrypted vote to the server.
- The server verifies the digital signature.
- The server decrypts the vote using its private key.
- The vote is stored in the SQLite database.

2.3 Architecture Diagram



3. Security Mechanisms

Property	Mechanism	Guarantee
Confidentiality	RSA Encryption	Only the server can read the vote
Authenticity	Digital Signature	Vote came from a legitimate voter
Integrity	SHA-256 Hashing	Vote was not altered in transit
Uniqueness	Voter ID Database	No double voting allowed

3.1 Confidentiality — RSA Encryption

The vote is encrypted using the server's public key. This ensures that only the server — holding the corresponding private key — can decrypt and read the vote content, protecting it from eavesdroppers.

3.2 Authenticity — Digital Signature

Each voter signs their encrypted vote using their own private key. The server verifies this signature using the voter's public key, confirming that the vote genuinely originated from the claimed sender.

3.3 Integrity — SHA-256 Hashing

The system applies SHA-256 hashing before signing the vote. This guarantees that any alteration to the data during transmission will be detected and rejected by the server.

3.4 Uniqueness — Double Voting Prevention

A SQLite database stores voter IDs. If a voter attempts to submit more than one vote, the system detects the duplicate entry and rejects the request immediately.

4. Implementation

4.1 Project Structure

```
.
├── client
│   ├── gui_client.py
│   ├── __init__.py
│   ├── __main__.py
│   ├── __pycache__
│   │   ├── gui_client.cpython-314.pyc
│   │   └── __init__.cpython-314.pyc
│   └── voter_client.py
├── crypto
│   ├── crypto_utils.py
│   ├── __init__.py
│   ├── __pycache__
│   │   ├── crypto_utils.cpython-314.pyc
│   │   ├── __init__.cpython-314.pyc
│   │   └── rsa_utils.cpython-314.pyc
│   └── rsa_utils.py
├── data
│   └── voting.db
├── main.py
├── __pycache__
├── readme.md
└── server
    ├── database.py
    ├── __init__.py
    ├── __main__.py
    ├── __pycache__
    │   ├── database.cpython-314.pyc
    │   ├── __init__.cpython-314.pyc
    │   └── voting_server.cpython-314.pyc
    └── voting_server.py
```

4.2 Technologies Used

Technology	Role
Python 3.x	Core language for all modules
Socket Programming	Client-server communication
RSA (cryptography lib)	Vote encryption & digital signature
SHA-256 (hashlib)	Data integrity hashing
SQLite3	Voter ID storage and vote counting
Tkinter	Graphical user interface

4.3 Installation and Execution

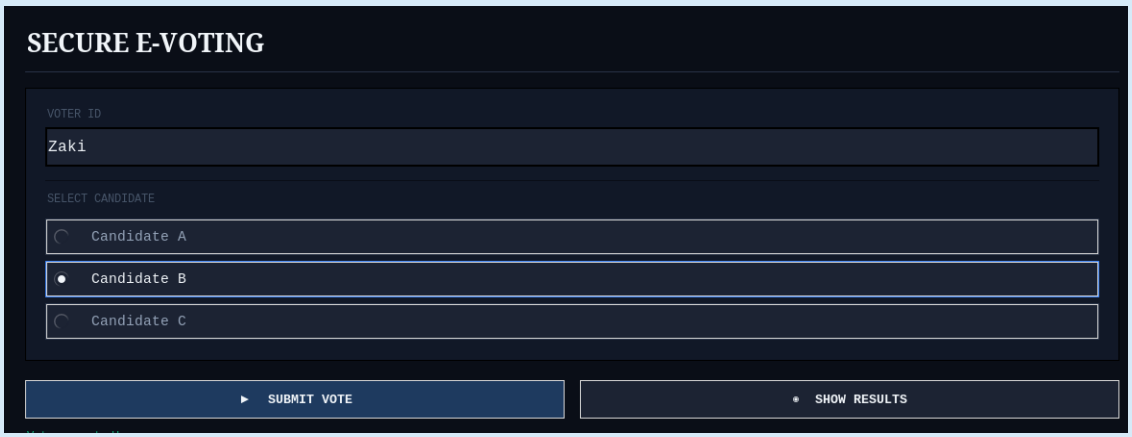
```
# Step 1: Navigate to project folder
cd secure_voting_system/

# Step 2: Run the application
python main.py

# The server will start automatically.
# The GUI will open — cast your vote!
```

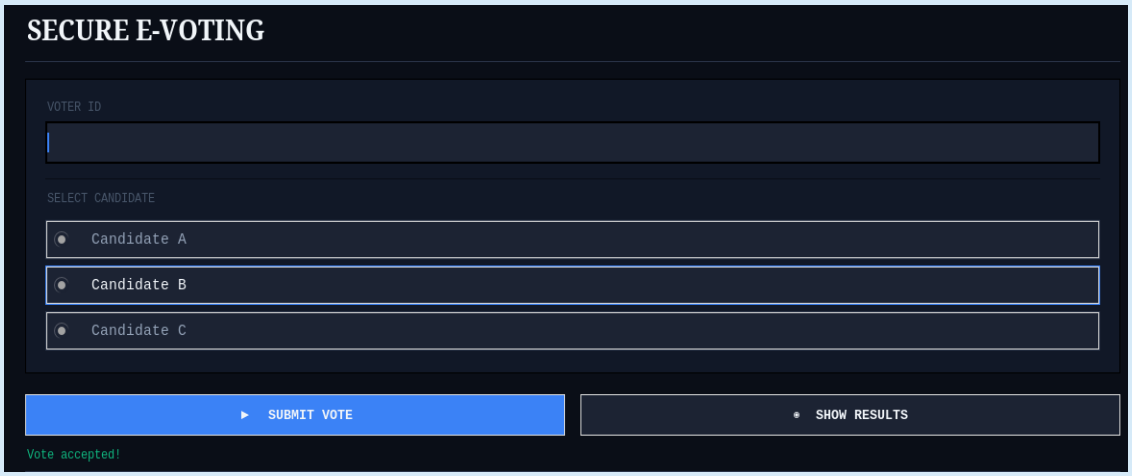
5. Tests and Validation

5.1 Test Cases Summary

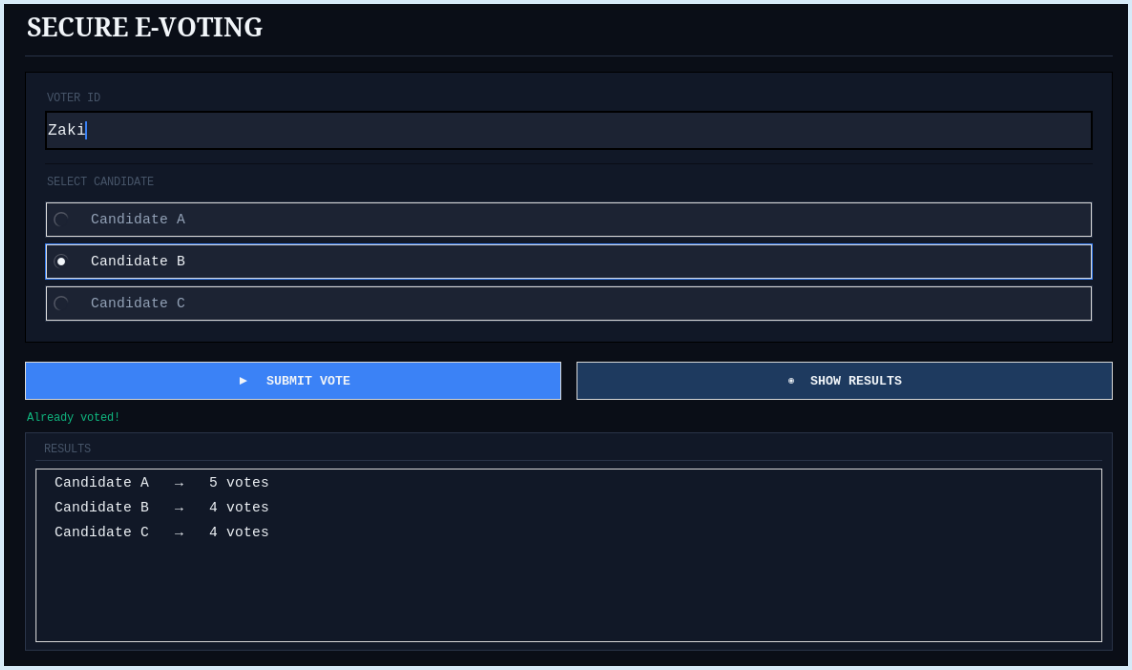
Test	Name	Input	Expected	Result
Test 1	Valid Vote	Valid ID + Candidate	Vote accepted	PASS
Test 2	Double Voting	Same ID twice	Already voted	PASS
Test 3	Invalid Signature	Tampered data	Signature rejected	PASS
Test 4	Results Display	Click Show Results	Correct tallies shown	PASS

5.2 Test Evidence — Screenshots





SECURE E-VOTING
VOTER ID
Zaki
SELECT CANDIDATE
Candidate A
Candidate B
Candidate C
SUBMIT VOTE
SHOW RESULTS
Already voted!


SECURE E-VOTING
VOTER ID
Zaki
SELECT CANDIDATE
Candidate A
Candidate B
Candidate C
SUBMIT VOTE
SHOW RESULTS
Already voted!
RESULTS
Candidate A → 5 votes
Candidate B → 4 votes
Candidate C → 4 votes

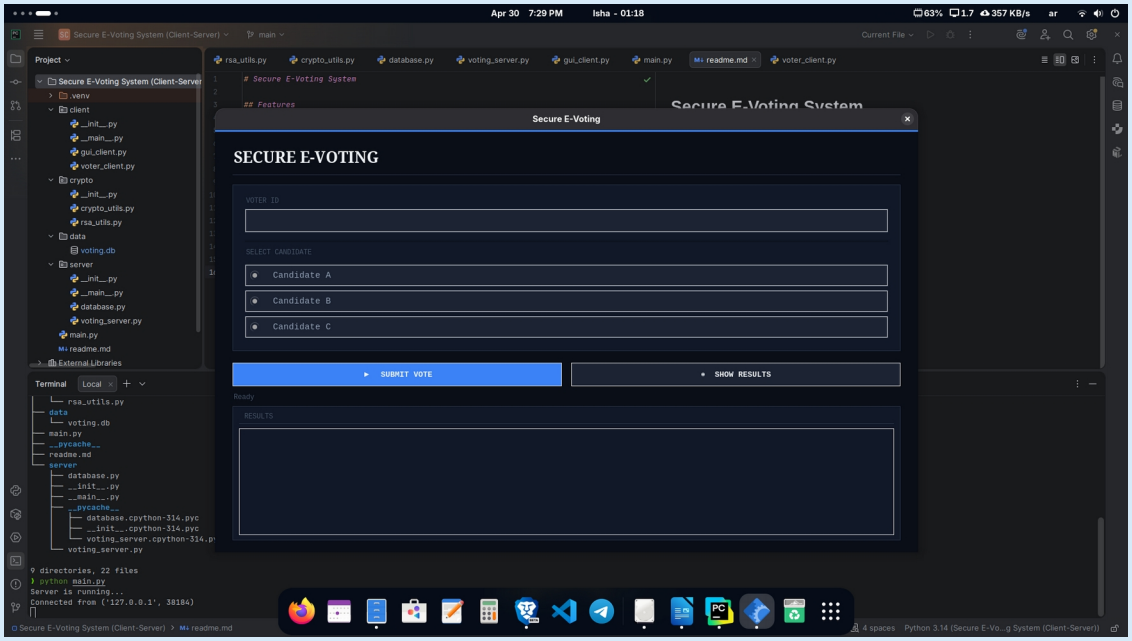
6. User Interface

The system includes a graphical user interface built with Python's Tkinter library. It was designed to be intuitive and accessible for any user without technical background.

6.1 Interface Features

- Input field for voter ID verification
- Candidate selection panel with radio buttons
- Vote submission button with real-time feedback
- Results display panel showing vote counts per candidate

6.2 GUI Screenshots



7. Contributions

7.1 Work Distribution

Student	Contribution
Brahmia Lokman	RSA encryption module, key generation
Meghlaoui Tarek	Client-server socket communication
Khennat Abdellah	GUI design (Tkinter)

7.2 External Tools and References

- ChatGPT — used for explanation and code assistance
- Python standard libraries (hashlib, socket, sqlite3, tkinter)
- cryptography library — for RSA key generation and operations

7.3 Adaptations and Improvements

The base system was improved through the following additions:

- Modular code structure separating crypto, networking, and GUI layers
- Graphical user interface for ease of use
- Real-time results display after voting closes
- Enhanced error handling and user feedback messages

8. Conclusion

This project demonstrates how cryptographic techniques can be applied to build a secure, reliable, and practical electronic voting system. By combining RSA encryption, digital signatures, SHA-256 hashing, and a database-backed uniqueness check, the system addresses all four core security properties: confidentiality, integrity, authenticity, and prevention of double voting.

The project also highlights the importance of modular software design and user experience in security-critical applications. The Tkinter-based interface ensures accessibility, while the underlying cryptographic stack provides robust protection.

This system reflects real-world applications of cybersecurity principles in distributed systems and serves as a solid foundation for more advanced implementations such as blockchain-based voting or zero-knowledge proofs.

Security Properties Achieved

Confidentiality Authenticity Integrity Non-repudiation